

JARVIS

Memory System Design

Architecture complète du système de mémoire — 3 niveaux, RAG pipeline, extraction, compression

Version : 1.0 · Dossier : 12_Memory_System / MSD-001 · Mai 2026

La mémoire est le composant le plus différenciant de JARVIS. Un chatbot oublie tout entre les sessions. JARVIS, lui, construit une représentation persistante de l'utilisateur — ses projets, ses préférences, ses habitudes, ses objectifs. Cette architecture à 3 niveaux est ce qui transforme un LLM générique en assistant personnel réellement utile.

1. Vue d'ensemble — Architecture à 3 niveaux

Le système de mémoire JARVIS est organisé en trois niveaux distincts, chacun avec une responsabilité, une technologie et une durée de vie différentes. Cette séparation est fondamentale : elle permet d'optimiser les coûts LLM, la pertinence du contexte et les performances globales du système.

NIVEAU 1 — Mémoire court terme

Technologie : Redis (in-memory)

Contient les 30 derniers messages de la session active. Rechargée à chaque requête LLM. Durée de vie : session active (expire après 2h d'inactivité). Ultra-rapide, zéro latence. C'est la fenêtre de conversation immédiate de JARVIS.

NIVEAU 2 — Mémoire épisodique

Technologie : PostgreSQL + pgvector (embeddings 1536 dimensions)

Contient les faits importants sur l'utilisateur : préférences, projets, événements, décisions, personnes clés. Persistante et permanente. Récupérée par recherche sémantique (cosine similarity) — JARVIS trouve les souvenirs les plus pertinents pour chaque requête, pas les plus récents.

NIVEAU 3 — Mémoire de travail

Technologie : PostgreSQL (relationnel structuré)

Contient l'état opérationnel du jour : tâches, agenda, emails importants, Morning Briefing, contexte des projets actifs. Mise à jour structurée. Injectée systématiquement dans le Morning Briefing et rafraîchie selon les actions de l'utilisateur.

Niveau	Technologie	Capacité	Durée de vie	Coût tokens injecté
Niveau 1 — Court terme	Redis	30 messages (~3000 tokens)	Session (2h inactivité)	~3000 tokens / req
Niveau 2 — Épisodique	PostgreSQL + pgvector	Illimitée (top-5 par req)	Permanente	~500 tokens / req
Niveau 3 — Travail	PostgreSQL relationnel	Tâches + agenda + briefing	Quotidienne/hebdo	~800 tokens / req
TOTAL injecté	—	—	—	~4300 tokens max / req

2. RAG Pipeline — Séquence détaillée

Le RAG (Retrieval Augmented Generation) est le processus qui enrichit chaque requête LLM avec le contexte mémoire pertinent. Voici la séquence exacte exécutée à chaque message utilisateur, avant l'appel au LLM.

Éta pe	Action	Composant	Durée cible
1	Réception du message utilisateur	API Gateway / FastAPI	< 10ms
2	Récupération Niveau 1 : 30 derniers messages depuis Redis	Redis GET session: {user_id}	< 5ms
3	Vectorisation de la question : conversion en embedding 1536D	OpenAI text-embedding-3-small ou Ollama nomic-embed	< 80ms
4	Recherche sémantique Niveau 2 : top-5 faits par cosine similarity dans pgvector	PostgreSQL pgvector ANN HNSW index	< 50ms
5	Récupération Niveau 3 : tâches actives + agenda du jour + projets en cours	PostgreSQL SELECT avec filtres user_id + date	< 20ms
6	Calcul du budget tokens total : N1 + N2 + N3 + system prompt	Token counter (tiktoken)	< 10ms
7	Compression si total > 6000 tokens : résumé des éléments les moins pertinents	LLM summarizer (Groq — rapide et gratuit)	< 500ms si déclenché
8	Construction du prompt final : system prompt + mémoire + question	Orchestrateur Python	< 10ms
9	Appel LLM via Router : Claude / Gemini Flash / Groq selon type de tâche	LLM Router	~800-3000ms
10	Streaming de la réponse vers le frontend	SSE / FastAPI StreamingResponse	Continu
11	Post-traitement async : extraction des nouveaux faits depuis la réponse	Fact Extractor (appel LLM léger Groq)	Arrière-plan < 1s
12	Sauvegarde des nouveaux faits en mémoire Niveau 2 avec embedding	PostgreSQL + pgvector INSERT	Arrière-plan < 200ms
13	Mise à jour du Niveau 1 : ajout des 2 nouveaux messages à Redis	Redis RPUSH + LTRIM (garde 30 max)	Arrière-plan < 5ms

3. Schéma de données — Tables mémoire

3.1 Table memories — Mémoire épisodique

PostgreSQL + extension pgvector

DDL — Table memories

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE memories (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  content     TEXT NOT NULL,
  embedding   vector(1536) NOT NULL,
  memory_type VARCHAR(50) NOT NULL,
  importance  FLOAT DEFAULT 0.5,
  source      VARCHAR(100),
  project_id  UUID REFERENCES projects(id),
  access_count INTEGER DEFAULT 0,
```

```
last_accessed TIMESTAMP,  
created_at  TIMESTAMP DEFAULT NOW(),  
updated_at  TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX ON memories USING hnsw (embedding vector_cosine_ops)  
WITH (m = 16, ef_construction = 64);  
  
CREATE INDEX ON memories (user_id, memory_type, importance DESC);
```

Types de mémoire (memory_type)

Type	Description	Exemples	Importance par défaut
preference	Préférences et habitudes de l'utilisateur	Aime le café le matin, travaille en musique	0.7
project	Informations sur un projet en cours	Projet JARVIS — stack Python/FastAPI/Next.js	0.8
person	Personnes importantes	Marc = associé sur le projet Alpha, contact Notion	0.7
fact	Fait biographique ou professionnel	Basé à Paris, fondateur startup IA depuis 2024	0.6
decision	Décision importante prise	Choix de Claude API comme LLM principal — mai 2026	0.9
goal	Objectif à court ou long terme	Lancer JARVIS MVP avant fin juillet 2026	0.8
task_context	Contexte lié à une tâche récurrente	Chaque lundi = standup équipe à 9h	0.6
emotion	État ou ressenti exprimé	A mentionné être stressé par le lancement	0.5

3.2 Table conversations — Historique sessions

Redis (court terme) + PostgreSQL (archivage long terme)

```
DDL — Table conversations (archivage PostgreSQL)  
CREATE TABLE conversations (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  project_id  UUID REFERENCES projects(id),  
  messages    JSONB NOT NULL DEFAULT '[]',  
  summary     TEXT,  
  tokens_used INTEGER DEFAULT 0,  
  llm_model   VARCHAR(100),  
  cost_usd    FLOAT DEFAULT 0,  
  started_at  TIMESTAMP DEFAULT NOW(),  
  ended_at    TIMESTAMP,  
  facts_extracted INTEGER DEFAULT 0  
);  
  
CREATE INDEX ON conversations (user_id, started_at DESC);  
CREATE INDEX ON conversations (project_id, started_at DESC);
```

Structure Redis — Mémoire court terme

Clé : session:{user_id}

Type : Redis List (LPUSH / LRANGE / LTRIM)

Format : JSON stringifié par message

```
{
  "role": "user" | "assistant",
  "content": "texte du message",
  "timestamp": "2026-05-13T10:23:00Z",
  "tokens": 142
}
```

Limite : 30 messages (LTRIM automatique)

TTL : 7200 secondes (2h d'inactivité → expire)

Restauration : si Redis expire → rechargement depuis PostgreSQL (dernière conversation)

4. Extraction automatique de faits

L'extraction de faits est le processus qui permet à JARVIS d'apprendre en permanence depuis les conversations. Après chaque réponse, un processus asynchrone analyse la conversation pour détecter les nouvelles informations à mémoriser.

4.1 Prompt d'extraction (Fact Extractor)

System prompt — Fact Extractor (modèle : Groq Llama 3.3 — rapide, gratuit)

Tu es un extracteur de faits pour un assistant IA personnel.

Analyse la conversation suivante et extrait UNIQUEMENT les faits nouveaux et importants sur l'utilisateur qui méritent d'être mémorisés à long terme.

RÈGLES STRICTES :

- Ne retourne RIEN si aucun fait nouveau n'est détecté
- Ignore les questions générales et les informations temporaires
- Retourne un JSON array de faits, chacun avec :
{ content, memory_type, importance (0.0-1.0), project_id? }

EXEMPLES de faits à extraire :

- 'Je travaille sur JARVIS avec FastAPI' → project, importance: 0.8
- 'Je préfère les réponses courtes' → preference, importance: 0.7
- 'Mon associé s'appelle Marc' → person, importance: 0.7

EXEMPLES de faits à IGNORER :

- 'Quelle heure est-il ?' → information temporaire
- 'Merci' → pas de fait mémorisable
- 'Calcule 12 x 8' → requête de calcul, pas de fait

4.2 Déduplication et mise à jour

Avant d'insérer un nouveau fait, le système vérifie si un fait similaire existe déjà en mémoire. Cette déduplication évite l'accumulation de faits redondants qui dégradent la qualité du contexte injecté.

- Vectorisation du nouveau fait → embedding 1536D
- Recherche des 3 faits les plus proches dans pgvector (cosine similarity)
- Si similarité > 0.92 → mise à jour du fait existant (content + importance + last_accessed)
- Si similarité entre 0.75 et 0.92 → nouveau fait créé avec référence au fait similaire
- Si similarité < 0.75 → nouveau fait indépendant créé
- access_count du fait existant incrémenté à chaque récupération → signal de pertinence

5. Stratégie de compression des tokens

La compression est le mécanisme de contrôle des coûts le plus important du système. Sans elle, chaque requête injecte de plus en plus de contexte jusqu'à dépasser la context window — ou jusqu'à ce que la facture devienne incontrôlable.

5.1 Budget tokens par requête

Composant	Tokens alloués	Priorité	Compressible
System prompt JARVIS (personnalité + règles)	~800	Fixe	Non
Profil utilisateur (nom, préférences clés)	~300	Haute	Non
Mémoire Niveau 1 — session courante	~3000	Haute	Partielle (résumé des plus anciens)
Mémoire Niveau 2 — faits épisodiques top-5	~500	Haute	Non (déjà sélectionnés)
Mémoire Niveau 3 — travail du jour	~800	Haute	Partielle (résumé si > 1000 tokens)
Question utilisateur	Variable (~50-500)	Critique	Non
BUDGET TOTAL CIBLE	< 6000 tokens	—	—
Déclenchement compression si	≥ 6000 tokens	—	Groq summarizer

5.2 Algorithme de compression

- Étape 1 : Calcul du total tokens avec tiktoken (CL100k_base pour Claude)
- Étape 2 : Si total < 5000 → aucune compression, injection directe
- Étape 3 : Si 5000 ≤ total < 6000 → compression douce : résumé des messages N1 au-delà des 15 derniers
- Étape 4 : Si total ≥ 6000 → compression agressive : résumé N1 complet + résumé des faits N2 les moins importants
- Étape 5 : Si total ≥ 7000 malgré compression → alerte log + injection des éléments les plus importants uniquement

- Étape 6 : Chaque compression est loggée avec les tokens économisés et le coût estimé évité

6. Sécurité de la mémoire

La mémoire de JARVIS contient les informations les plus personnelles et sensibles de l'utilisateur. Sa protection est non négociable.

Risque	Protection	Implémentation
Accès cross-utilisateur	RLS (Row Level Security) PostgreSQL	ALTER TABLE memories ENABLE ROW LEVEL SECURITY — politique user_id stricte
Fuite en cas de dump DB	Chiffrement AES-256 du champ content	Fernet Python avant INSERT, déchiffrement avant lecture
Injection via mémoire	Sanitisation avant injection dans le prompt	Échappement des caractères spéciaux, longueur max par fait : 500 chars
Accumulation non contrôlée	Limite de 500 faits par utilisateur	Suppression des faits les moins importants et les plus anciens au-delà du seuil
Export non autorisé	Export uniquement via endpoint authentifié	GET /api/memory/export → JWT requis + log de l'action
Suppression sur demande	RGPD — droit à l'oubli complet	DELETE CASCADE sur user_id → suppression de toutes les mémoires et embeddings

7. Monitoring et observabilité

Le système mémoire doit être observé en continu pour détecter les dérives de coût, les anomalies de qualité et les problèmes de performance avant qu'ils n'impactent l'expérience utilisateur.

- Tokens injectés par requête → loggé dans llm_logs avec décomposition N1/N2/N3
- Nombre de compressions déclenchées par jour → alerte si > 30% des requêtes déclenchent une compression
- Qualité des faits extraits → score de similarité moyen des requêtes pgvector (cible > 0.6)
- Temps de récupération mémoire → alerte si latence pgvector > 200ms (indique besoin de re-indexation HNSW)
- Croissance de la mémoire par utilisateur → graphe du nombre de faits dans le temps
- Faits les plus accédés → top-10 par utilisateur → signal de pertinence pour améliorer l'extraction
- Coût estimé de la mémoire → tokens_used × coût_par_token, agrégé par utilisateur et par mois